

Финальная демонстрация Defect Detection System

Этот гайд нужен для защиты проекта. Он показывает, как пройти по всем требованиям, что запускать, какие файлы показывать и как объяснять архитектуру.

0. Перед началом

Перейти в финальную ветку:

```
git switch feat/final-project-compliance
git status --short --branch
```

Ожидаемое состояние:

```
## feat/final-project-compliance...origin/feat/final-project-compliance
```

Базовые проверки:

```
source .venv/bin/activate
make lint
python -m pytest tests
dvc status
docker compose config
```

Что сказать:

Перед демонстрацией проверяется качество проекта: lint, тесты, DVC-статус и
валидность Docker Compose. Это показывает, что код, данные и инфраструктурные
конфиги находятся в согласованном состоянии.

1. Датасет и базовая модель

Файлы:

```
data/raw.dvc
data/raw/
models/best_model.pth
notebooks/defect-detection.ipynb
src/defect_detection/modeling/
```

Проверить состояние DVC:

```
dvc status
```

Если данных или модели нет локально:

```
dvc pull
```

Что сказать:

В проекте используется Severstal Steel Defect Detection. Датасет и модель
вынесены в DVC, потому что это тяжелые артефакты. Git хранит код и DVC
metadata, а сами данные и модель управляются через DVC.

Почему так реализовано:

- Git не подходит для больших бинарных файлов.

- DVC позволяет воспроизводимо подтянуть нужные версии данных и модели.
- Модель **models/best_model.pth** используется inference-сервисом.

2. Git workflow, conventional commits, DVC

Показать историю:

```
git log --oneline --max-count 12
```

Что сказать:

Работа велась через feature branch. Коммиты оформлены в стиле conventional commits: **feat, fix, docs, style, chore**. Это делает историю понятной и пригодной для review.

Проверить DVC:

```
dvc status
```

Ожидаемо:

Data and pipelines are up to date.

3. Структура проекта

Показать структуру:

```
ls
```

Главные директории:

```
data/      данные под DVC
models/    модельные артефакты
notebooks/ baseline experiments
src/       production-код
tests/     тесты
reports/   отчеты
monitoring/ Prometheus, Grafana, reference stats
k8s/       Kubernetes и Argo CD
storage/   SQLite runtime storage
```

Что сказать:

Структура близка к Cookiecutter Data Science: данные, модели, эксперименты, production-код, отчеты и инфраструктура разделены. Так проект проще поддерживать и проверять.

4. MLflow tracking

Запустить MLflow:

```
docker compose up -d mlflow
```

Открыть:

```
http://127.0.0.1:5050
```

Файлы:

```
src/defect_detection/modeling/train.py
docker-compose.yml
mlflow-data/
```

Запуск training endpoint:

```
MLFLOW_TRACKING_URI=http://127.0.0.1:5050 \
python -m defect_detection.modeling.train
```

Что сказать:

MLflow используется для трекинга экспериментов: параметры, метрики, artifacts и регистрация модели. В Docker Compose данные MLflow сохраняются в **mlflow-data**, поэтому не пропадают после перезапуска контейнера.

5. CI/CD

Файл:

```
.github/workflows/ci.yml
```

Показать workflow:

```
sed -n '1,220p' .github/workflows/ci.yml
```

Что сказать:

GitHub Actions запускает проверки на PR и push: установка зависимостей, Ruff lint, pytest и Docker build. Это CI-часть. CD-часть для Kubernetes реализована через Argo CD.

Локальные аналоги CI:

```
make lint
python -m pytest tests
docker build -t defect-detection-api:local .
```

6. FastAPI, OpenAPI, Docker

Запустить API локально:

```
uvicorn defect_detection.api.main:app --reload
```

Открыть:

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/health
http://127.0.0.1:8000/ready
```

Файлы:

```
src/defect_detection/api/main.py
src/defect_detection/api/routes.py
src/defect_detection/api/schemas.py
Dockerfile
docker-compose.yml
```

Что сказать:

FastAPI выбран потому, что автоматически дает OpenAPI-документацию и хорошо
подходит для inference API. Dockerfile собирает production-like образ.
В Dockerfile используется CPU-only PyTorch, чтобы не тянуть CUDA-пакеты в
локальном и CI-окружении.

Запустить через Docker Compose:

```
docker compose up --build api
```

7. Drift, Prometheus, Grafana

Запустить полный локальный стек:

```
docker compose up --build
```

Открыть:

API: <http://127.0.0.1:8000>
Prometheus: <http://127.0.0.1:9090>
Grafana: <http://127.0.0.1:3000>

Grafana credentials:

admin / admin

Файлы:

```
src/defect_detection/monitoring/drift.py
src/defect_detection/api/metrics.py
monitoring/reference_stats.json
monitoring/reference_target_distribution.json
monitoring/prometheus.yml
monitoring/prometheus.minikube.yml
monitoring/grafana/dashboards/defect-detection.json
```

Что сказать:

Data drift считается по статистикам входного изображения относительно
reference stats. Target drift считается по распределению предсказанных классов
относительно baseline distribution. Concept drift считается по feedback:
если true classes расходятся с prediction, растёт mismatch rate.

Показать drift endpoint:

```
http://127.0.0.1:8000/drift/status
```

Prometheus queries:

```
defect_predictions_total
defect_data_drift_value
defect_target_drift_value
defect_concept_drift_value
defect_feedback_total
```

Если API запущен в Minikube, а Prometheus/Grafana запускаются локально, нужно использовать отдельный monitoring compose:

```
kubectrl port-forward svc/defect-detection-api 8000:8000
docker compose stop api mflow
docker compose -f docker-compose.monitoring.yml up -d
```

В этом режиме **monitoring/prometheus.minikube.yml** настраивает Prometheus на **host.docker.internal:8000**. Это адрес локального port-forward к FastAPI service в Minikube. Проверить, что Prometheus действительно видит Kubernetes API:

```
http://127.0.0.1:9090/targets
```

В targets должен быть **defect-detection-api-minikube** со статусом **UP**.

8. Drift reports

API должен быть запущен.

Сгенерировать отчет:

```
python -m defect_detection.monitoring.report \
--status-url http://127.0.0.1:8000/drift/status \
--output-dir reports/drift
```

Файлы:

```
src/defect_detection/monitoring/report.py
reports/drift/README.md
reports/drift/
```

Что сказать:

Отчет генерируется из текущего /drift/status . Это Markdown snapshot
состояния дрейфа: data drift, target drift, concept drift, thresholds,
current values и baseline values.

9. Web UI

Открыть:

```
http://127.0.0.1:8000/ui
http://127.0.0.1:8000/ui/predictions
http://127.0.0.1:8000/ui/experiments
```

Английская версия:

```
http://127.0.0.1:8000/ui?lang=en
```

Файлы:

```
src/defect_detection/api/ui.py
src/defect_detection/api/templates/
src/defect_detection/api/static/app.js
src/defect_detection/api/translations.py
src/defect_detection/api/storage.py
```

Что показать:

- загрузить изображение;
- получить prediction;
- увидеть overlay маски;
- открыть историю предсказаний;
- отправить feedback;

- увидеть drift status;
- нажать retraining;
- увидеть статус retraining job;
- переключить RU/EN.

Что сказать:

UI сделан внутри FastAPI как простой server-rendered интерфейс. Для учебного проекта это разумнее, чем отдельный frontend-сервис. Переводы лежат в **translations.py**, поэтому русский и английский интерфейс поддерживаются из одного места.

10. SQLite runtime storage

Файл базы:

storage/app.db

Файлы кода:

src/defect_detection/api/storage.py
src/defect_detection/api/main.py
docker-compose.yml
.gitignore

Что хранится:

predictions
feedback
retraining_jobs

Что сказать:

Раньше история предсказаний, feedback и retraining jobs жили бы только в памяти процесса API. После рестарта они терялись бы. SQLite добавлен как легкая локальная база без отдельного сервера. Для учебного проекта это разумный компромисс между in-memory state и полноценным PostgreSQL.

Почему файл не коммитится:

storage/app.db - runtime state, зависящий от локальных запусков. В Git хранится только **storage/.gitkeep**, чтобы директория существовала.

11. Retraining job

Endpoints:

POST /retrain
GET /retrain/jobs
GET /retrain/status/{job_id}

Файлы:

src/defect_detection/api/retraining.py
src/defect_detection/api/routes.py

```
src/defect_detection/api/storage.py
src/defect_detection/api/static/app.js
```

Что сказать честно:

Сейчас retraining реализован как demo job: он создает задачу, запускает
background task, пишет статус в SQLite и логирует demo metrics в MLflow. Это
показывает MLOps workflow: trigger -> job -> MLflow -> status -> UI.
В UI отображаются последняя job, история jobs и ссылка на MLflow run, если
demo run успешно залогирован.

Почему так:

Полноценное переобучение U-Net и горячая замена модели внутри API сильно
увеличивают сложность и требуют GPU/долгих вычислений, validation gate,
promotion в Model Registry и safe reload модели. Для финальной учебной
демонстрации важнее показать управляемый workflow.

Что потребовалось бы для production-grade слоя:

- реальный training job вместо demo metrics;
- сравнение метрик новой модели с текущей;
- validation gate;
- promotion в MLflow Model Registry;
- безопасная перезагрузка модели в API;
- хранение статусов model versions;
- лучше запускать retraining как Kubernetes Job или Airflow DAG, а не внутри FastAPI процесса.

12. Kubernetes и Minikube

Файлы:

```
k8s/api/deployment.yaml
k8s/api/pvc.yaml
k8s/api/service.yaml
k8s/mlflow/deployment.yaml
k8s/mlflow/pvc.yaml
k8s/mlflow/service.yaml
```

Запустить Minikube:

```
minikube start
```

Собрать image внутри Minikube Docker:

```
eval $(minikube docker-env)
docker build -t defect-detection-api:local .
eval $(minikube docker-env -u)
```

Деплой MLflow:

```
kubectl apply -f k8s/mlflow/
kubectl get pods
```

```
kubectl get svc
minikube service mlflow
```

Для ссылок на MLflow runs из UI:

```
kubectl port-forward svc/mlflow 5000:5000
```

Деплой API:

```
kubectl apply -f k8s/api/
kubectl get pods
kubectl get svc
kubectl port-forward svc/defect-detection-api 8000:8000
```

Открыть:

```
API UI:    http://127.0.0.1:8000/ui
API docs:  http://127.0.0.1:8000/docs
MLflow:    http://127.0.0.1:5000
Prometheus: http://127.0.0.1:9090
Grafana:   http://127.0.0.1:3000
```

Что сказать:

Kubernetes-манифесты показывают production-like деплой. Deployment управляет
pod, Service дает сетевой доступ, PersistentVolumeClaim сохраняет runtime
storage для API и MLflow. Minikube используется как локальный
Kubernetes-кластер. API отправляет demo retraining runs во внутренний service
http://mlflow:5000 . Prometheus/Grafana можно оставить в Docker Compose:
Prometheus ходит к API в Minikube через локальный port-forward.

13. Argo CD

Файл:

```
k8s/argocd/application.yaml
```

Что он делает:

```
GitHub repository -> k8s/api -> Kubernetes cluster
```

Установка Argo CD:

```
kubectl create namespace argocd
kubectl apply --server-side -n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl get pods -n argocd
```

Применить Application:

```
kubectl apply -f k8s/argocd/application.yaml
kubectl get applications -n argocd
```

Если демонстрация идет до merge в **main**, временно направить Argo CD на текущую ветку:

```
kubectl patch application defect-detection-api -n argocd \
--type merge \
-p '{"spec":{"source":{"targetRevision":"feat/final-project-compliance"}}}'
```

Открыть UI:


```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

<https://127.0.0.1:8080>

Что сказать:

Argo CD реализует GitOps: Git становится source of truth. В этом проекте Argo CD следит за **k8s/api** и деплоит FastAPI inference service. MLflow деплоится отдельными Kubernetes-манифестами через **kubectl**.

Почему Argo CD только для FastAPI нормально:

Требование говорит про CD с помощью Argo CD в Kubernetes или Minikube. Главный production-facing сервис проекта - FastAPI inference service. Поэтому Argo CD для API закрывает требование. MLflow в проекте выступает как tracking-сервис и деплоится отдельными manifests.

14. README

Файлы:

README.md
README.en.md

Что сказать:

Основной README на русском, потому что защита проекта проходит на русском. Английская версия находится в **README.en.md**. В README описаны запуск, Docker, MLflow, мониторинг, drift reports, Kubernetes, Argo CD и проверки.

Финальный маршрут демонстрации

Рекомендуемый порядок:

- 1 Показать **git status**, **git log**, структуру проекта.
- 2 Показать **dvc status**, объяснить данные и модель.
- 3 Запустить **make lint** и **pytest**, объяснить quality gate.
- 4 Для Docker-демо запустить **docker compose up --build**.
- 5 Открыть **/docs**, **/health**, **/ready**.
- 6 Открыть **/ui**, загрузить изображение, показать prediction.
- 7 Открыть **/ui/predictions**, показать историю из SQLite.
- 8 Отправить feedback, показать concept drift.
- 9 Открыть **/drift/status**.
- 10 Открыть Prometheus, показать метрики.
- 11 Открыть Grafana, показать dashboard.
- 12 Сгенерировать drift report.
- 13 Для Kubernetes-демо запустить Minikube, применить **k8s/mlflow/** и **k8s/api/**, открыть API через **kubectl port-forward**.

- 14 Для мониторинга Minikube API запустить **`docker compose -f docker-compose.monitoring.yml up -d`**.
- 15 Открыть Prometheus targets и Grafana dashboard.
- 16 Нажать retraining в UI, показать статус job и MLflow.
- 17 Показать **`k8s/`** и **`k8s/argocd/application.yaml`**.
- 18 Завершить README и объяснением, что требования закрыты.

Короткая формулировка для защиты

Это MLOps-система для defect detection: данные и модель версионизируются через
DVC, эксперименты логируются в MLflow, inference работает через FastAPI и
Docker, runtime-состояние хранится в SQLite, качество и дрейф мониторяются
через Prometheus/Grafana, отчеты по дрейфу генерируются в Markdown, Web UI
позволяет делать inference, смотреть историю, отправлять feedback и запускать
demo retraining workflow. Для production-like деплоя есть Kubernetes-манифесты
и Argo CD Application для GitOps-доставки FastAPI-сервиса в Minikube.